

Lecture 10: Instructions

CMPE I20: Computer Organization & Architecture

Olivia Weng

Logistics

- HW2 released!
 - Due October 1st
- HW1 submission troubleshooting
 - We will try our best to grade your HW1
 - HW2 and beyond: your responsibility that the Gradescope autograder runs successfully
 - Reach out to staff if any issues before the deadline

How to design an ISA?

- Goal: Want an ISA that can be designed and executed efficiently in hardware

How to design an ISA?

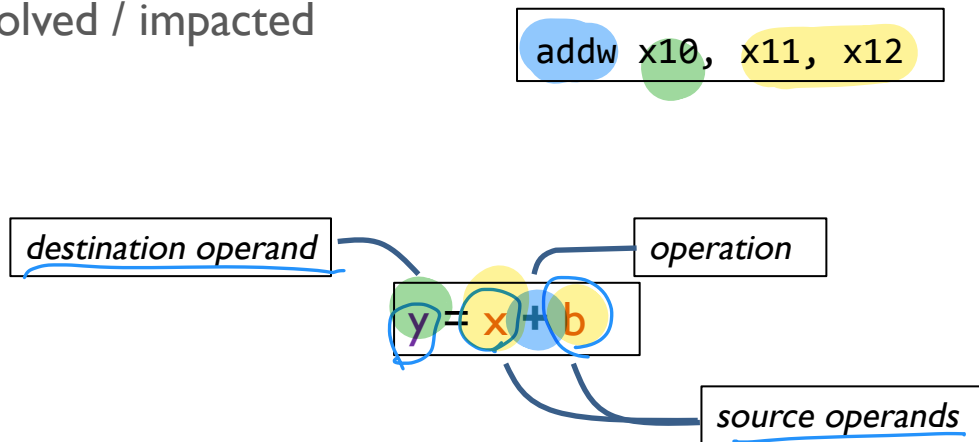
- Goal: Want an ISA that can be designed and executed efficiently in hardware
- By efficiency, we usually focus on
 - ↓ Latency — time
 - ↑ Throughput — # stuff / s
 - Resource consumption ↓
 - Power consumption ↓

There are always tradeoffs!

How to design an ISA?

We need

- **operations**: what an instruction does
- **operands**: specify what data is involved / impacted
- **instruction format**



How to design an ISA?

- operations

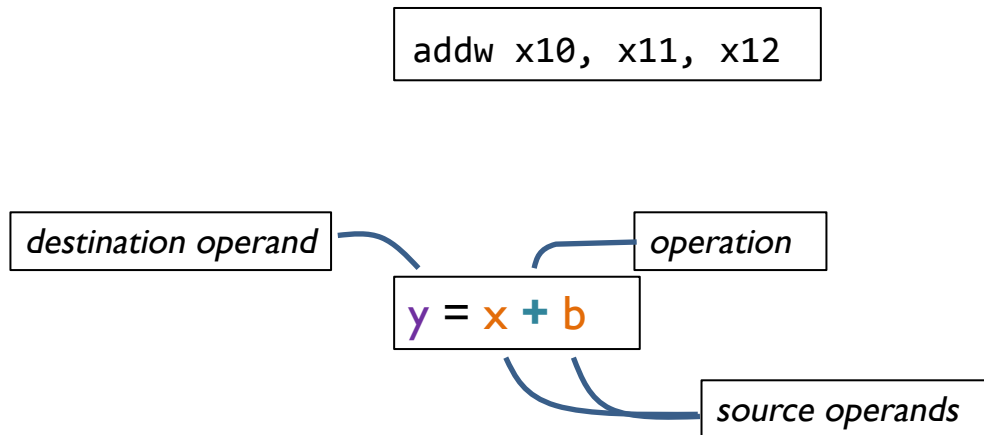
- how many?
- which ones?

- operands

- how many?
- location
- types
- how to specify?

- instruction format

- size
- how many formats?



Practice problem



Your architecture supports 16 instructions and 16 registers (0-15).

You have fixed width instructions which are 16 bits. *addw x10, x11, x12*

How many register operands can you specify (explicitly) in an add instruction?

$$16 - 4 = 12 \text{ bits left}$$

$$\begin{array}{rcl} 4 \text{ bits} & & \\ 2 & = & 16 \text{ values} \\ \text{---} & & \end{array}$$

4 bits instr
4 bits reg

$$\frac{12}{4} = \boxed{\leq 3 \text{ register operands}}$$

$$2^5 = 32$$

$$14 - 5 = 9 \text{ bits left}$$

Practice problem

5 bits

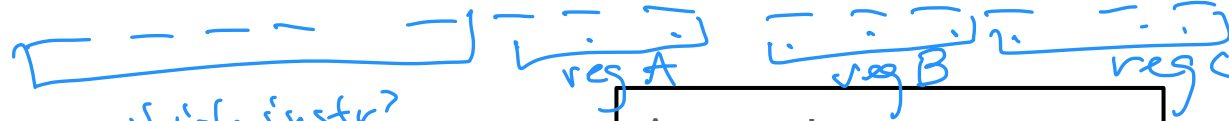
3 bits $2^3 = 8$

Your architecture supports 32 instructions and 8 registers (0-7).

You have fixed width instructions which are 14 bits.

$$\frac{9 \text{ bits left}}{3 \text{ bits}} = 3 \text{ reg.}$$

How many register operands can you specify (explicitly) in an add instruction?



Which instr?
What operation?
add?
sub?
or?

- A. ≤ 1
- B. ≤ 2
- C. ≤ 3
- D. ≤ 4
- E. None of the above

ISA design principles

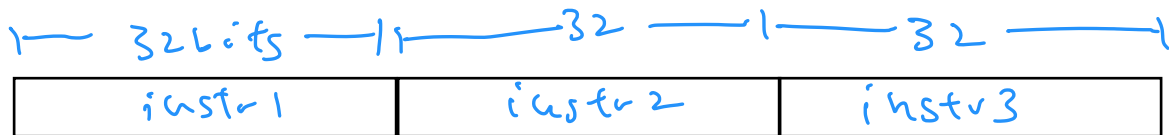
The textbook describes several design principles for a good ISA:

1. **Simplicity** favors regularity
2. Make the **common** case **fast**
3. **Smaller** is faster
4. Good design demands **good compromises**

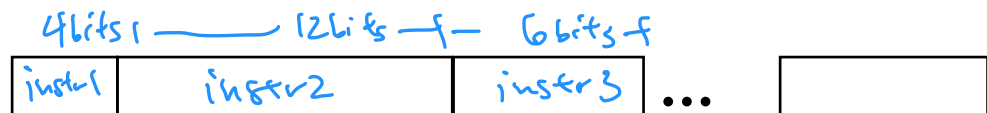


How long should an instruction be?

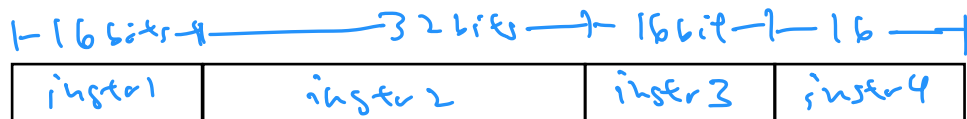
- Fixed



- Variable



- Hybrid



`add x10, x11, x12`

Instruction length tradeoffs

- **Fixed-length** instructions (RISC-V)
 - easy fetch and decode
 - simplify pipelining and parallelism
- **Variable-length** instructions (Intel 80x86, VAX)
 - multi-step fetch and decode
 - much more flexible and compact instruction set.
- **Hybrid** instructions (ARM)
 - Middle ground

Instruction length tradeoffs

- **Fixed-length** instructions (RISC-V)

- easy fetch and decode
- simplify pipelining and parallelism

→ All RISC-V instructions are **32 bits** long.

DPI: Simplicity favors **regularity**.

- **Variable-length** instructions (Intel 80x86, VAX)

- multi-step fetch and decode
- much more flexible and compact instruction set.

- **Hybrid** instructions (ARM)

- Middle ground

Instruction format

- Instruction format: what each bit means
 - Having many different instruction formats
 - complicates decoding
 - uses more instruction bits to specify the format
 - could allow us to take full advantage of a variable-length ISA

RISC-V instruction format

- 4 types of instruction formats
 - R-type: register
 - I-type: immediate - constant
 - S/B-type: Store/Branch
 - U/J-type: Upper-Immediate/Jump

RISC-V instruction format

- 4 types of instruction formats

- R-type: register

- I-type: immediate

- S/B-type: Store/Branch

- U/J-type: Upper-Immediate/Jump

DP4: Good design demands good compromises.

which function?
→ add, sub, xor...

7 bits	5 bits	5 bits	3 bits	5 bits	7 bits
funct7	rs2	rs1	funct3	rd	op
imm _{11:0}		rs1	funct3	rd	op
imm _{11:5}	rs2	rs1	funct3	imm _{4:0}	op
imm _{12,10:5}	rs2	rs1	funct3	imm _{4:1,11}	op
imm _{31:12}				rd	op
imm _{20,10:1,11,19:12}				rd	op
20 bits				5 bits	7 bits

R-Type

I-Type

S-Type

B-Type

U-Type

J-Type

which format

RISC-V instruction format

DP4: Good design demands **good compromises**.

- 4 types of instruction formats

- R-type**: register

- I-type**: immediate

- S/B-type**: Store/Branch

- U/J-type**: Upper-Immediate/Jump

7 bits	5 bits	5 bits	3 bits	5 bits	7 bits
funct7	rs2	rs1	funct3	rd	op
imm _{11:0}		rs1	funct3	rd	op
imm _{11:5}	rs2	rs1	funct3	imm _{4:0}	op
imm _{12,10:5}	rs2	rs1	funct3	imm _{4:1,11}	op
imm _{31:12}				rd	op
imm _{20,10:1,11,19:12}				rd	op
20 bits				5 bits	7 bits

R-Type

I-Type

S-Type

B-Type

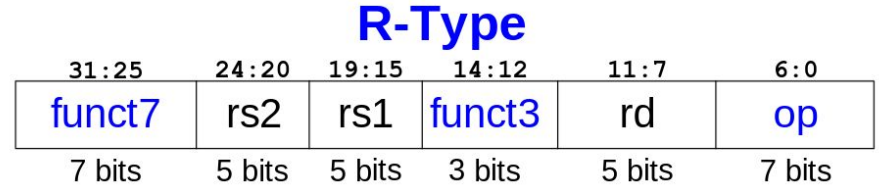
U-Type

J-Type

The **opcode** tells machine which format.

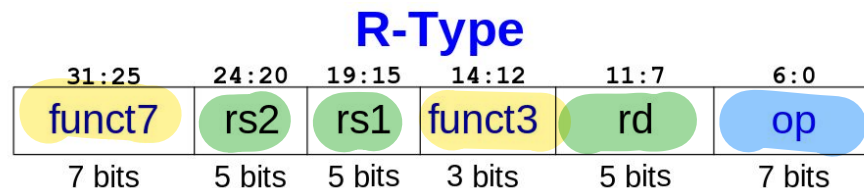
Example instruction encoding

- Register-type



Example instruction encoding

- Register-type
- 3 register operands
 - rs1, rs2: source registers
 - rd: destination register



- op: operation code (opcode)
- funct7, funct3:
 - the function combined with opcode tells machine exactly what operation to perform

Example instruction encoding

C code	RISC-V assembly code
<code>a = b + c;</code>	<code>add a, b, c</code>

Example instruction encoding

C code	RISC-V assembly code
a = b + c;	add a, b, c

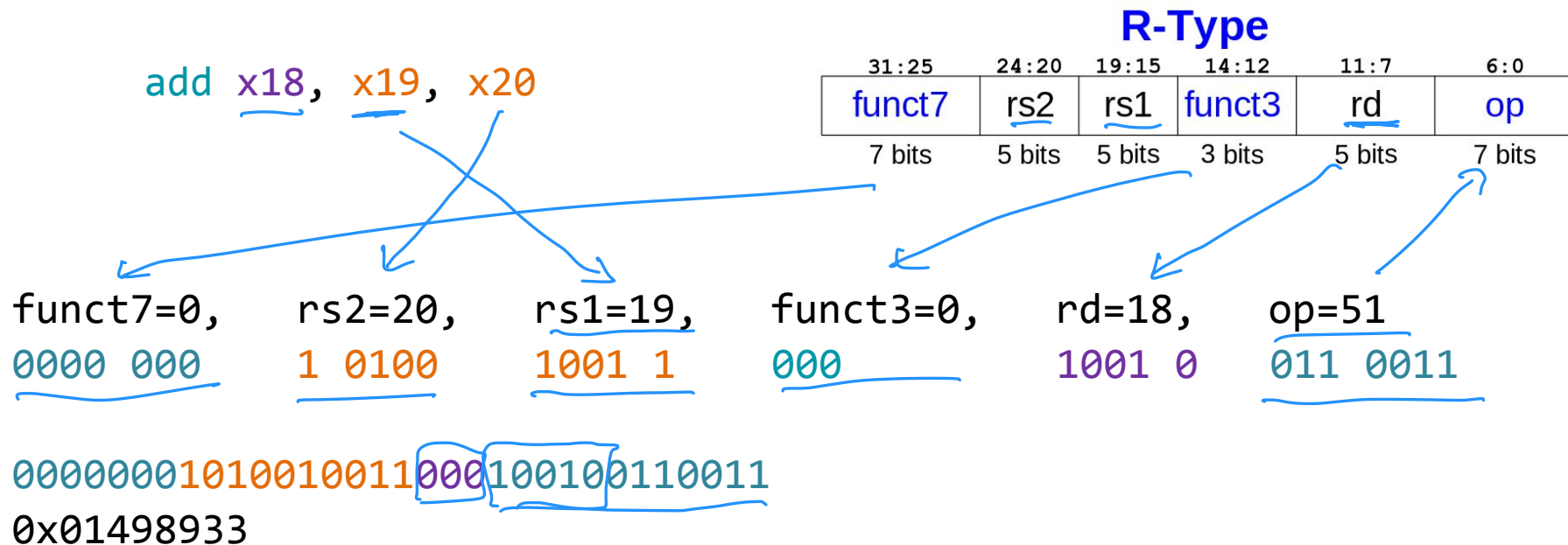
add x18, x19, x20

R-Type

31:25	24:20	19:15	14:12	11:7	6:0
funct7	rs2	rs1	funct3	rd	op
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

Example instruction encoding

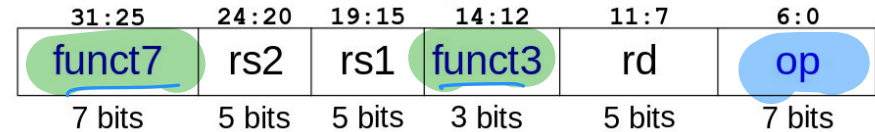
C code	RISC-V assembly code
<code>a = b + c;</code>	<code>add a, b, c</code>



Implications of RISC-V instruction format

- What is the maximum number of unique R-type operations RISC-V encodes?
 - Only 1 opcode for R-type

R-Type



- A. 8
 - B. 64
 - C. 512
 - D. 1024
 - E. None of the above

$$2^7 \cdot 2^3 = 2^{10} = 1024$$